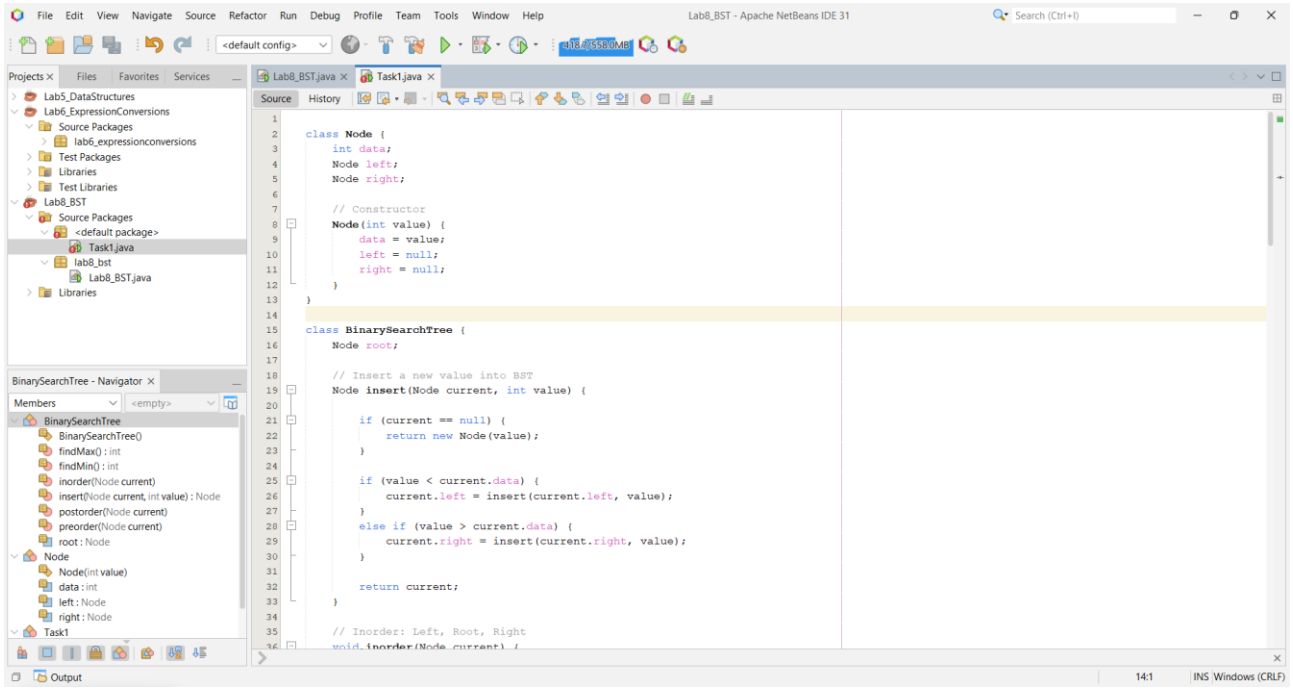
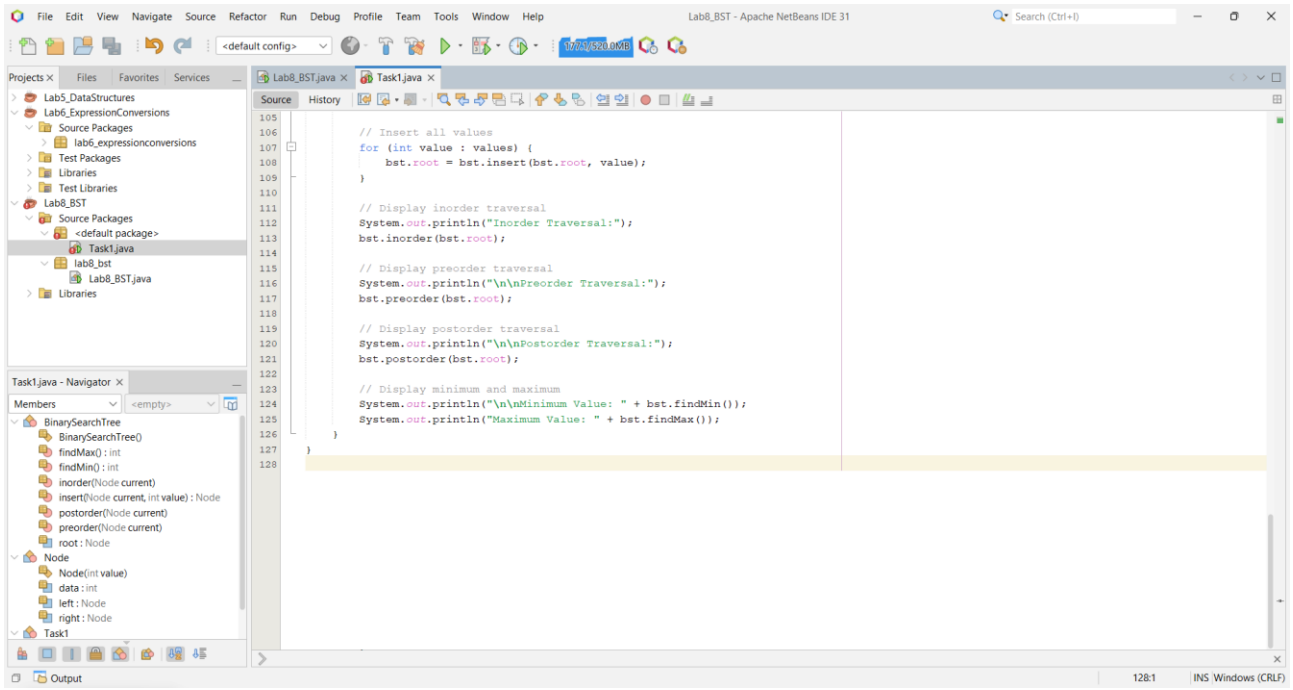
RIPHAH
INTERNATIONAL UNIVERSITY

DATA STRUCTURE

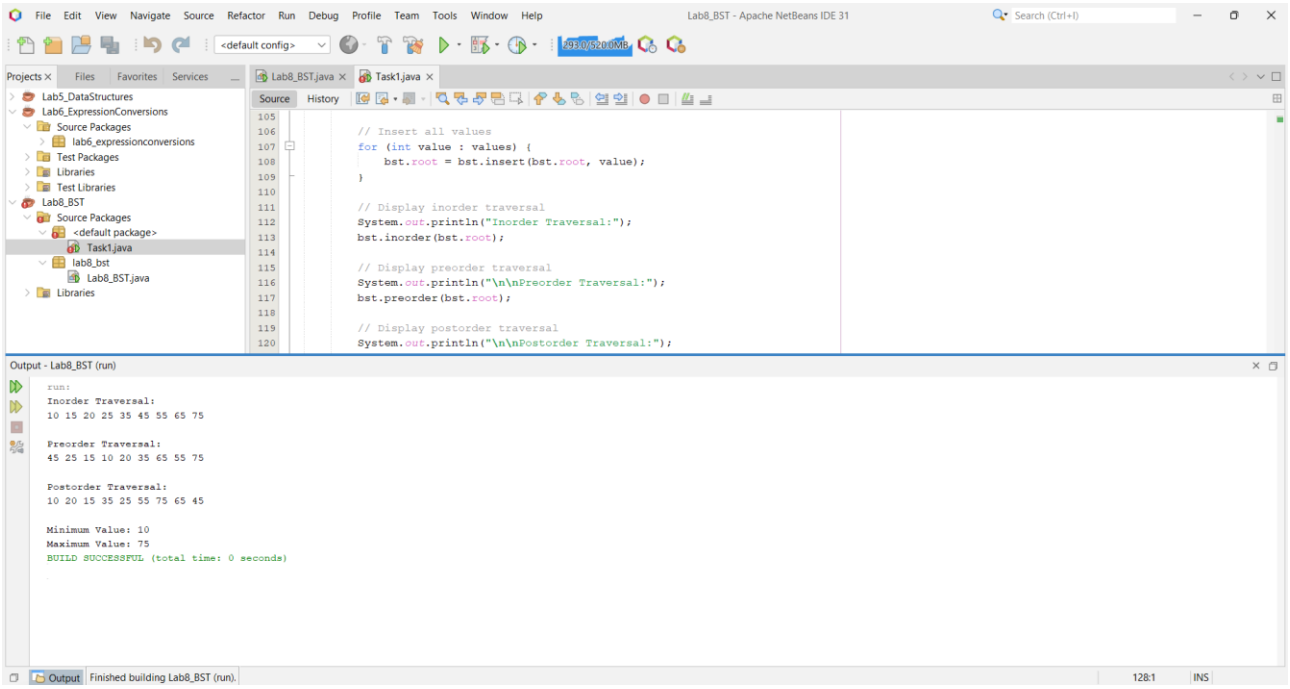
NAME	Qayam Raza
SAPID	60332
INSTRUCTOR	Sir Junaid khan
DATE	04/11/2026
LAB NO	8

Task No 1 — BST Construction and Traversal

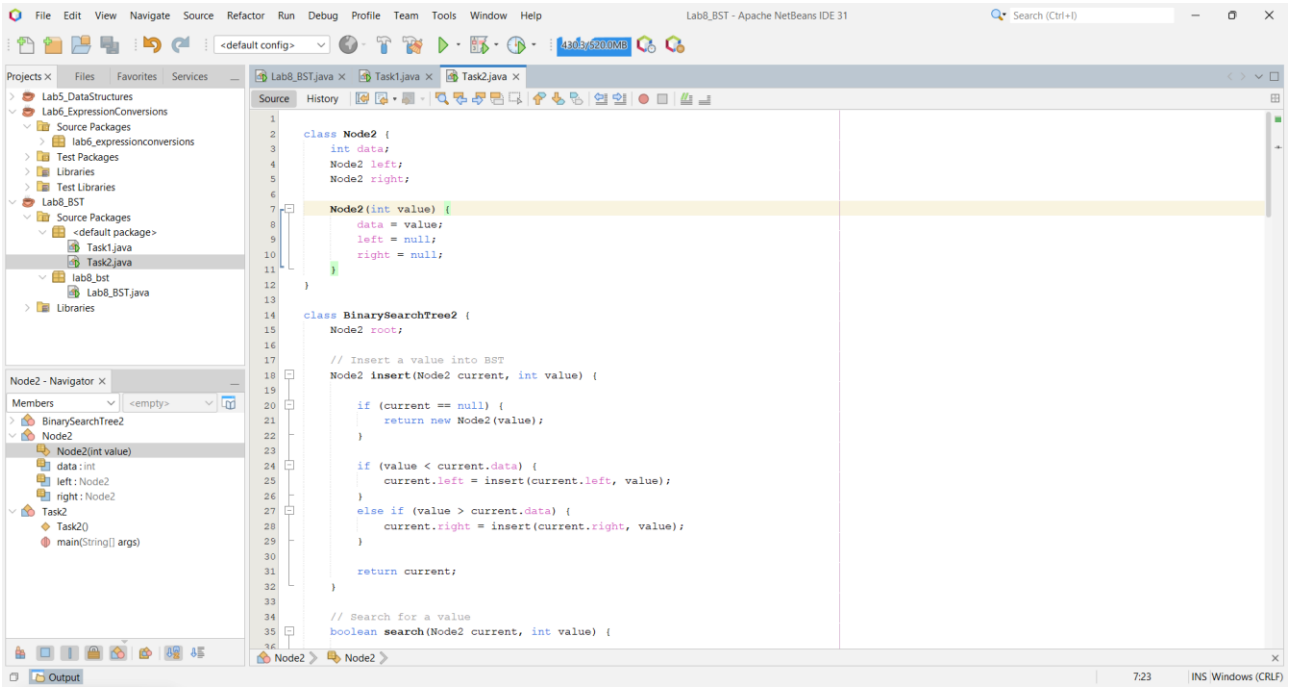


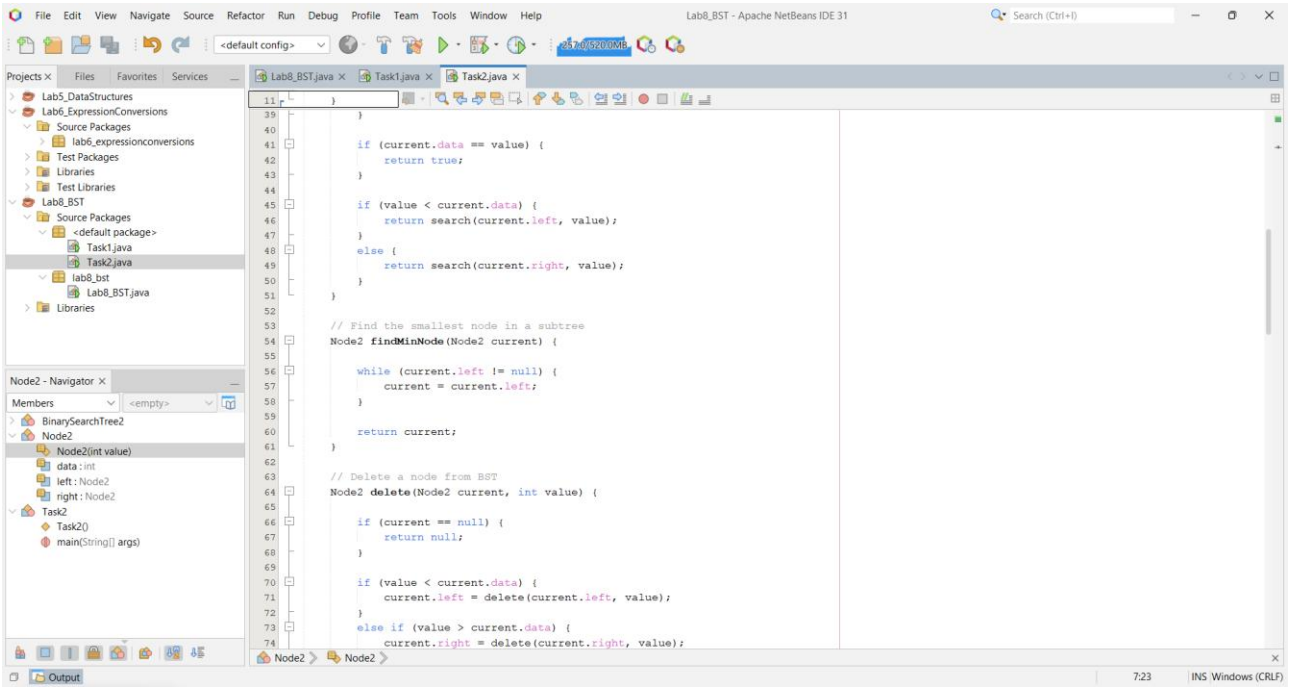


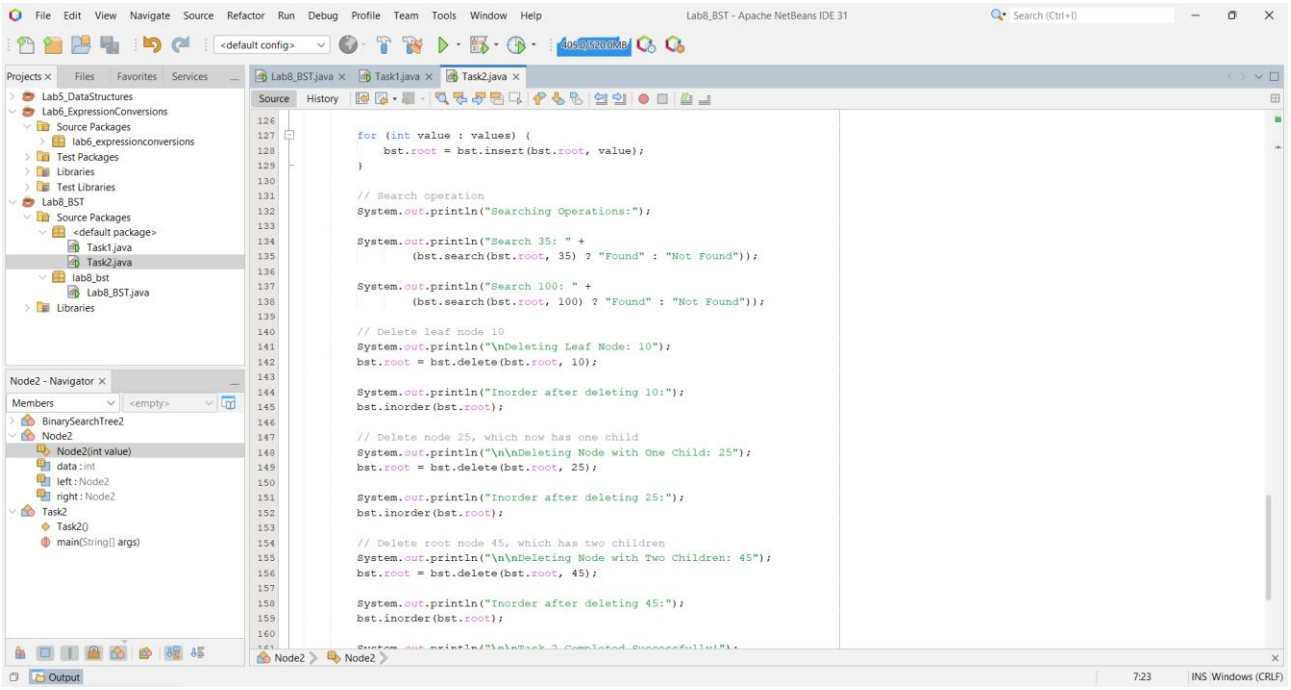
Task No 1 — Output



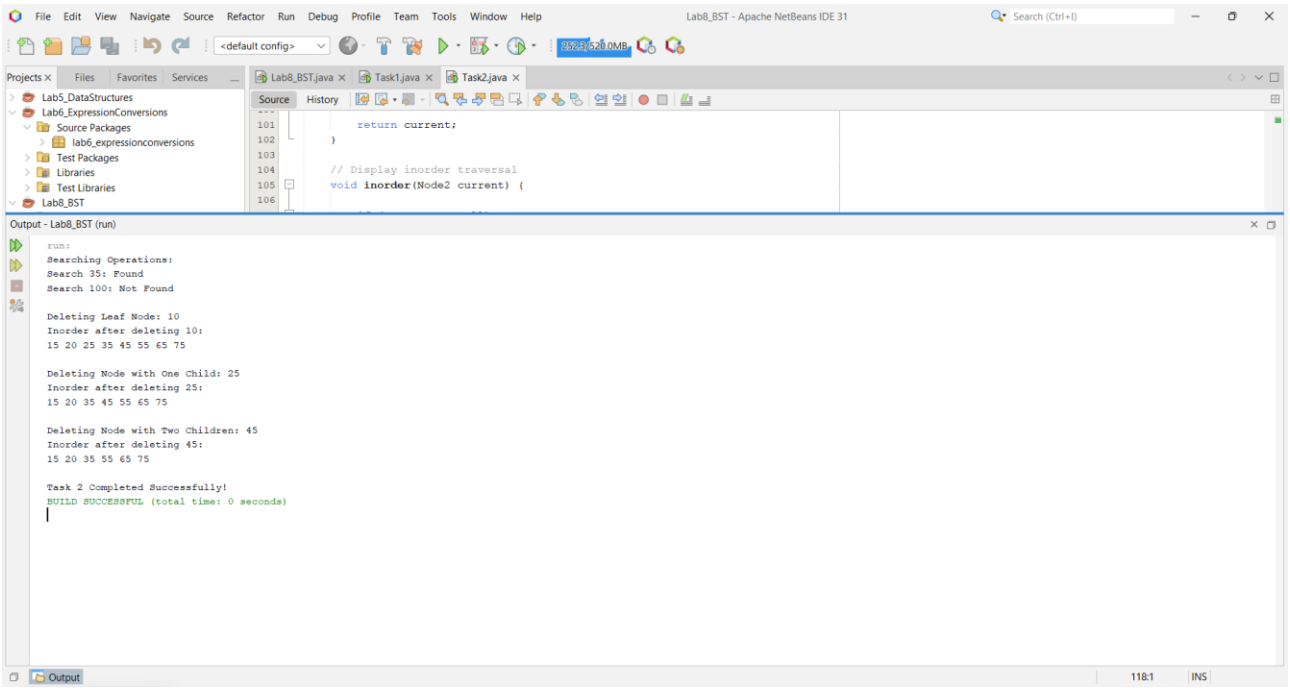
Task No 2 — Searching and Deletion




File Edit View Navigate Source Refactor Run Debug Profile Team Tools Window Help
Lab8_BST - Apache NetBeans IDE 31
Search (Ctrl+I)
Projects
Files
Favorites
Services
Lab5_DataStructures
Lab6_ExpressionConversions
Source Packages
lab6_expressionconversions
Test Packages
Libraries
Test Libraries
Lab8_BST
Source Packages
<default package>
Task1.java
Task2.java
lab8_bst
Lab8_BST.java
Libraries
Node2 - Navigator
Members
<empty>
BinarySearchTree2
Node2
Node2(int value)
data : int
left : Node2
right : Node2
Task2
Task2()
main(String[] args)
Lab8_BST.java
Task1.java
Task2.java
Node2
Node2
Output
7:23
INS Windows (CRLF)


File Edit View Navigate Source Refactor Run Debug Profile Team Tools Window Help
Lab8_BST - Apache NetBeans IDE 31
Search (Ctrl+I)
Projects
Files
Favorites
Services
Lab5_DataStructures
Lab6_ExpressionConversions
Source Packages
lab6_expressionconversions
Test Packages
Libraries
Test Libraries
Lab8_BST
Source Packages
<default package>
Task1.java
Task2.java
lab8_bst
Lab8_BST.java
Libraries
Node2 - Navigator
Members
<empty>
BinarySearchTree2
Node2
Node2(int value)
data : int
left : Node2
right : Node2
Task2
Task2()
main(String[] args)
Lab8_BST.java
Task1.java
Task2.java
Source
History
Node2
Node2
Output
7:23
INS Windows (CRLF)

Task No 2 — Output



Task 2 – Explanation of BST Structure Changes After Each Deletion

1. Deletion of Node 10 (Leaf Node)

Node 10 is a leaf node because it has no children. When we delete it, it is simply removed from the BST. Its parent node, 15, remains in the tree and now has only one child, node 20. The rest of the BST structure remains unchanged.

2. Deletion of Node 25

After deleting node 10, node 25 still has two children: 15 and 35. Therefore, we use the inorder successor method to delete it. The inorder successor of 25 is 35 because it is the smallest value in its right subtree. Node 35 replaces node 25, and its original position is removed. The BST property remains maintained.

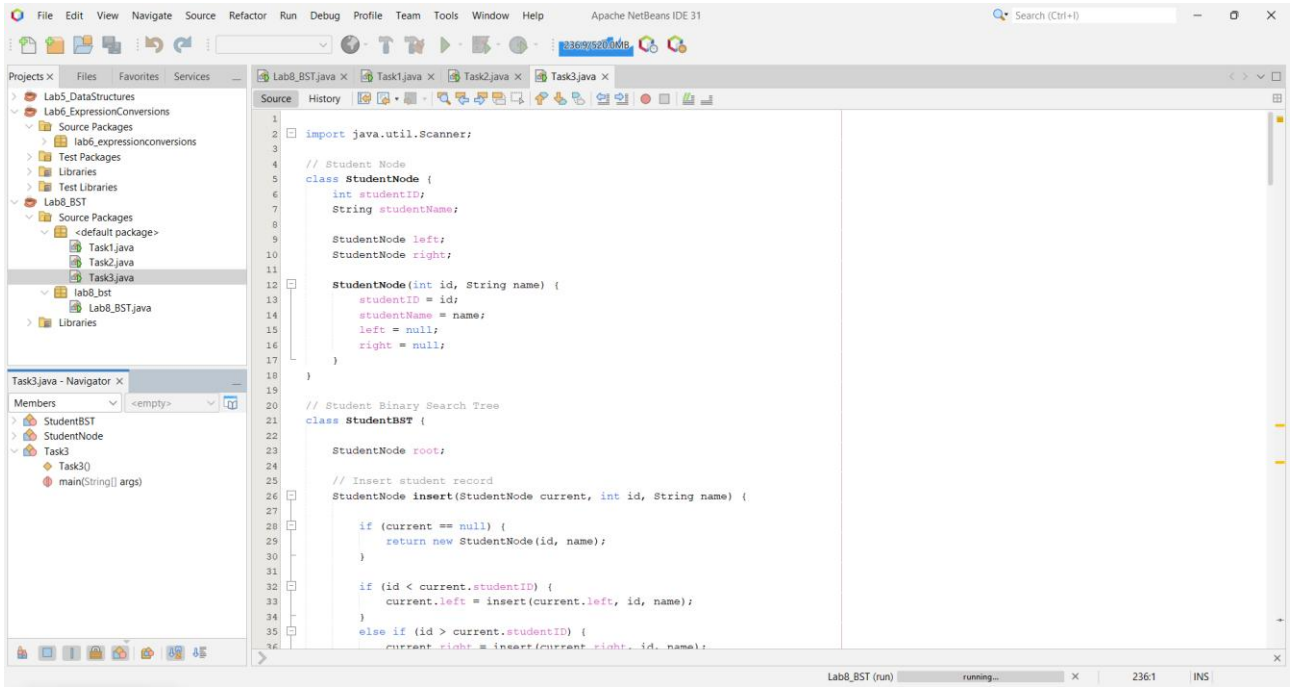
3. Deletion of Node 45 (Two Children)

Node 45 is the root node and has two children. To delete it, we find its inorder successor, which is 55 (the smallest value in the right subtree). We replace 45 with 55 and remove the original node 55 from its previous position. The resulting BST maintains the correct ordering of values.

Conclusion:

Deletion in a Binary Search Tree depends on the number of children a node has. A leaf node is removed directly, a node with one child is replaced by its child, and a node with two children is replaced by its inorder successor or predecessor.

Task No 3 — Student Record Management Using BST



```java
import java.util.Scanner;

// Student Node
class StudentNode {
    int studentID;
    String studentName;

    StudentNode left;
    StudentNode right;

    StudentNode(int id, String name) {
        studentID = id;
        studentName = name;
        left = null;
        right = null;
    }
}

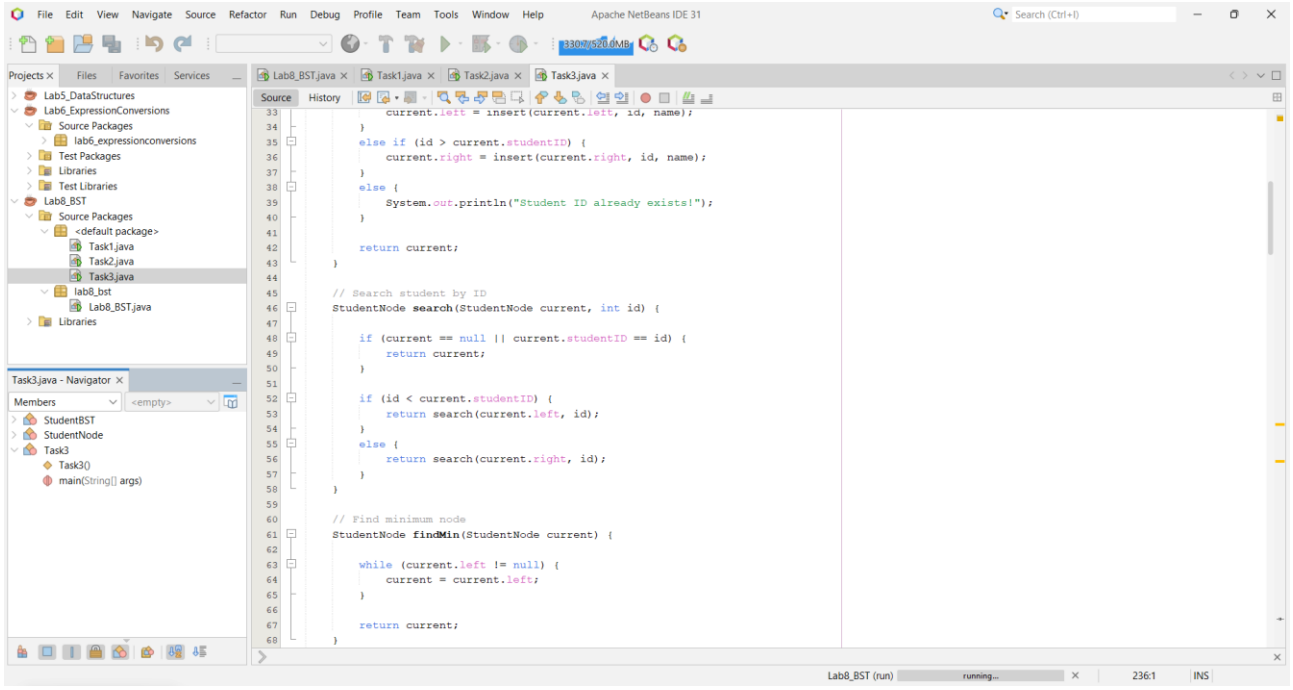
// Student Binary Search Tree
class StudentBST {

    StudentNode root;

    // Insert student record
    StudentNode insert(StudentNode current, int id, String name) {

        if (current == null) {
            return new StudentNode(id, name);
        }

        if (id < current.studentID) {
            current.left = insert(current.left, id, name);
        }
        else if (id > current.studentID) {
```



```java
            current.left = insert(current.left, id, name);
        }
        else if (id > current.studentID) {
            current.right = insert(current.right, id, name);
        }
        else {
            System.out.println("Student ID already exists!");
        }

        return current;
    }

    // Search student by ID
    StudentNode search(StudentNode current, int id) {

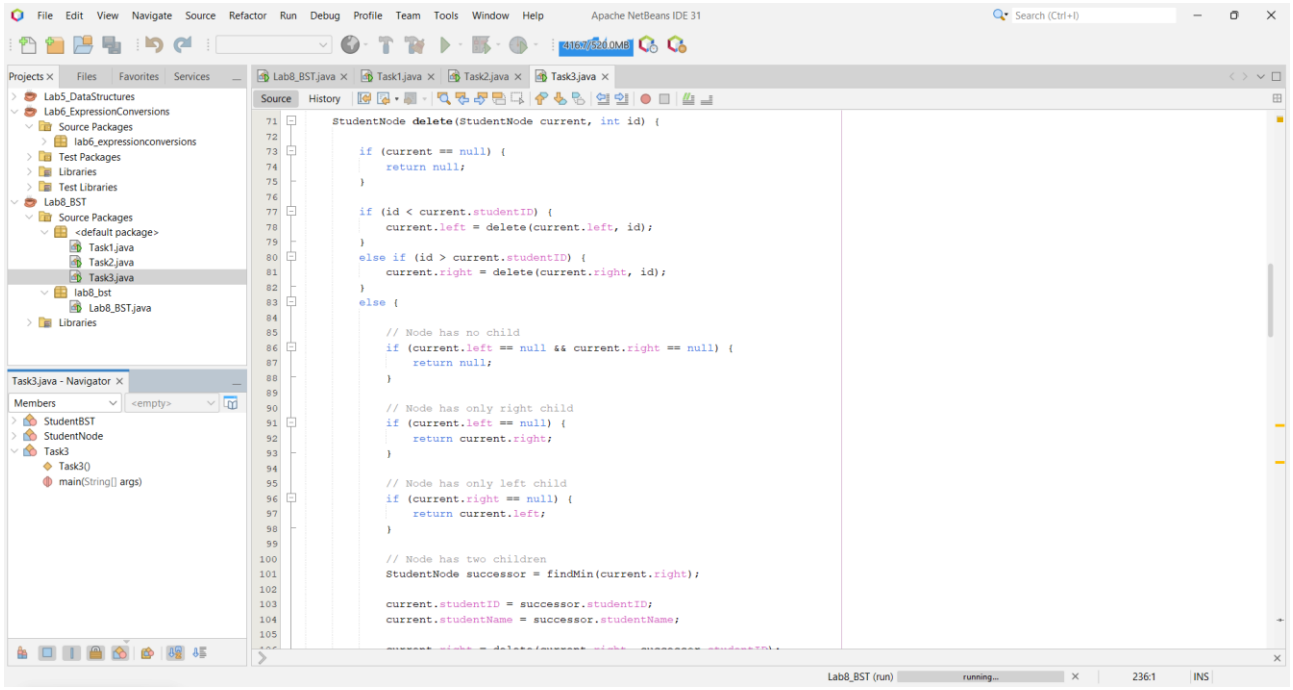
        if (current == null || current.studentID == id) {
            return current;
        }

        if (id < current.studentID) {
            return search(current.left, id);
        }
        else {
            return search(current.right, id);
        }
    }

    // Find minimum node
    StudentNode findMin(StudentNode current) {

        while (current.left != null) {
            current = current.left;
        }

        return current;
    }
```



```
    StudentNode delete(StudentNode current, int id) {

        if (current == null) {
            return null;
        }

        if (id < current.studentID) {
            current.left = delete(current.left, id);
        }
        else if (id > current.studentID) {
            current.right = delete(current.right, id);
        }
        else {

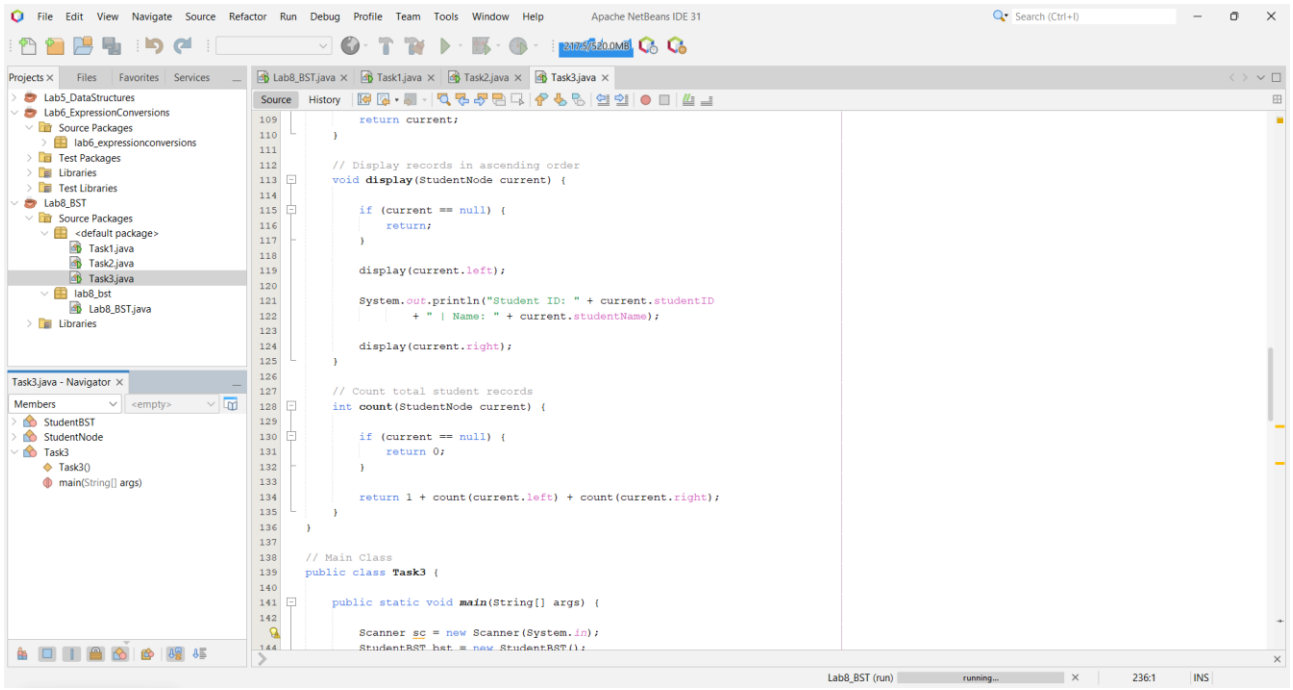
            // Node has no child
            if (current.left == null && current.right == null) {
                return null;
            }

            // Node has only right child
            if (current.left == null) {
                return current.right;
            }

            // Node has only left child
            if (current.right == null) {
                return current.left;
            }

            // Node has two children
            StudentNode successor = findMin(current.right);

            current.studentID = successor.studentID;
            current.studentName = successor.studentName;
```



```
            return current;
        }

        // Display records in ascending order
        void display(StudentNode current) {

            if (current == null) {
                return;
            }

            display(current.left);

            System.out.println("Student ID: " + current.studentID
                    + " | Name: " + current.studentName);

            display(current.right);
        }

        // Count total student records
        int count(StudentNode current) {

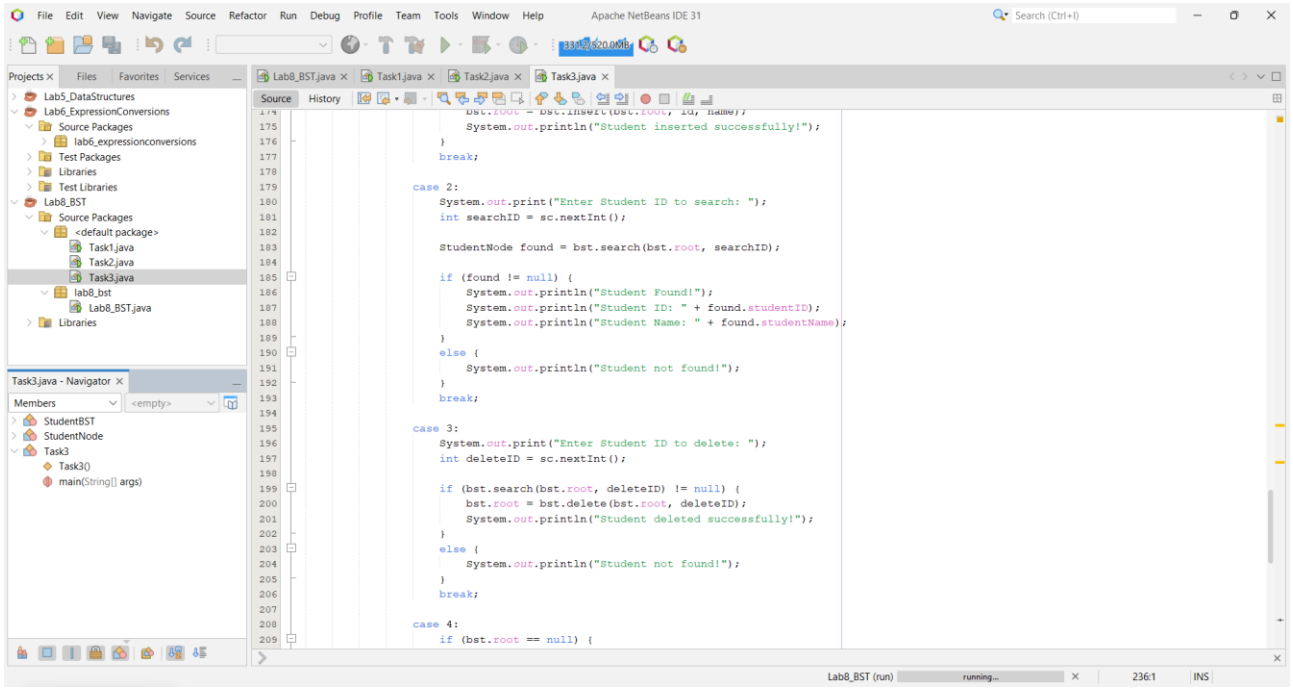
            if (current == null) {
                return 0;
            }

            return 1 + count(current.left) + count(current.right);
        }
    }

    // Main Class
    public class Task3 {

        public static void main(String[] args) {

            Scanner sc = new Scanner(System.in);
            StudentBST bst = new StudentBST();
```



```
                    System.out.println("Student inserted successfully!");
                }
                break;

            case 2:
                System.out.print("Enter Student ID to search: ");
                int searchID = sc.nextInt();

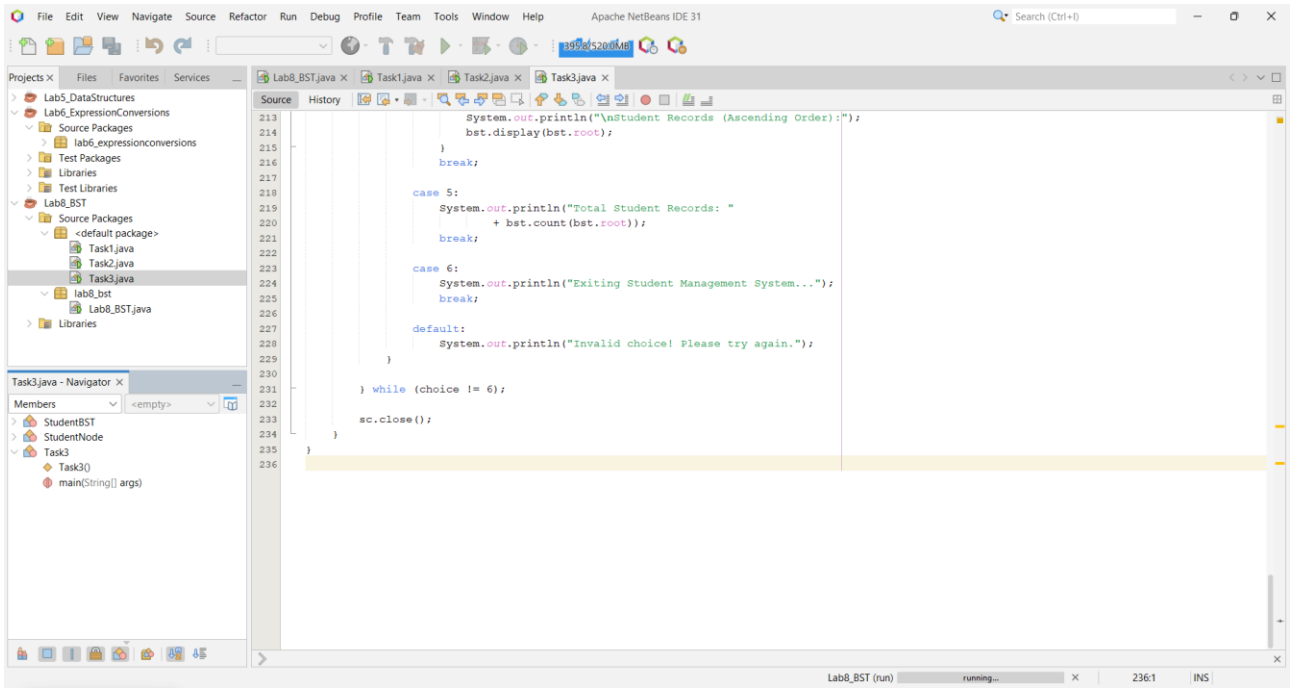
                StudentNode found = bst.search(bst.root, searchID);

                if (found != null) {
                    System.out.println("Student Found!");
                    System.out.println("Student ID: " + found.studentID);
                    System.out.println("Student Name: " + found.studentName);
                }
                else {
                    System.out.println("Student not found!");
                }
                break;

            case 3:
                System.out.print("Enter Student ID to delete: ");
                int deleteID = sc.nextInt();

                if (bst.search(bst.root, deleteID) != null) {
                    bst.root = bst.delete(bst.root, deleteID);
                    System.out.println("Student deleted successfully!");
                }
                else {
                    System.out.println("Student not found!");
                }
                break;

            case 4:
                if (bst.root == null) {
```



```
                    System.out.println("\nStudent Records (Ascending Order):");
                    bst.display(bst.root);
                }
                break;

            case 5:
                System.out.println("Total Student Records: "
                        + bst.count(bst.root));
                break;

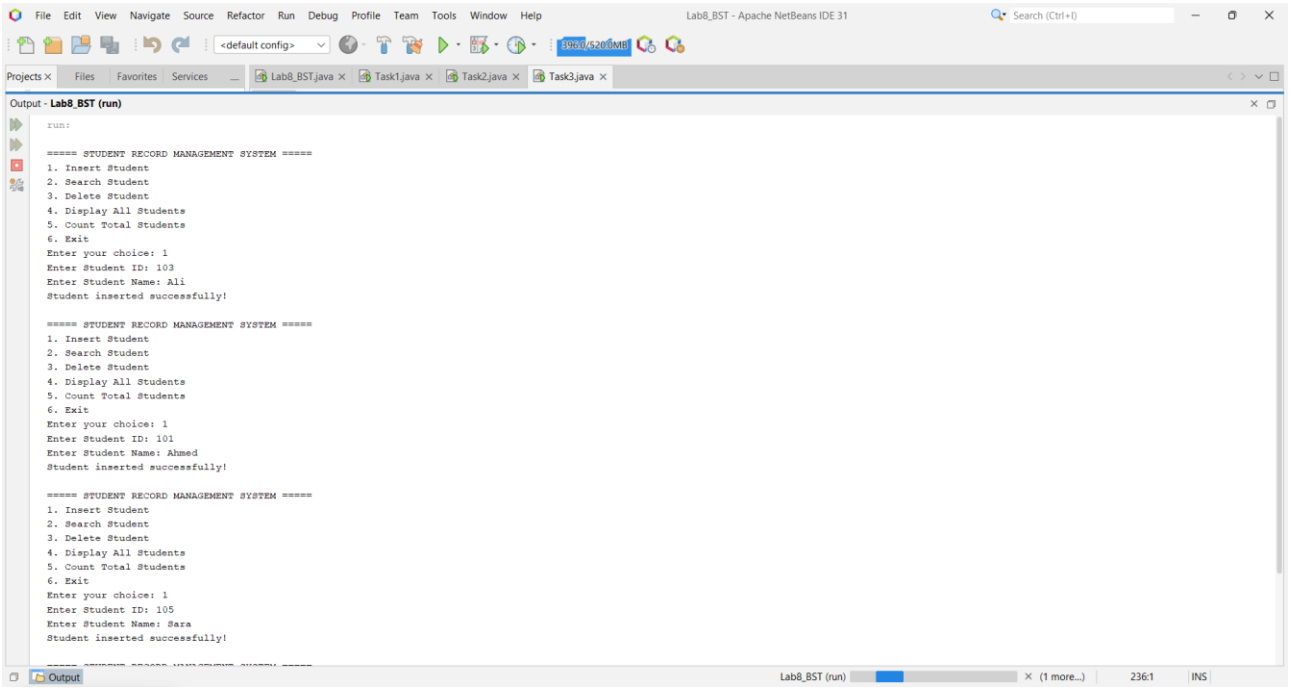
            case 6:
                System.out.println("Exiting Student Management System...");
                break;

            default:
                System.out.println("Invalid choice! Please try again.");
        }

    } while (choice != 6);

    sc.close();
  }
}
```

Task No 3 — Output

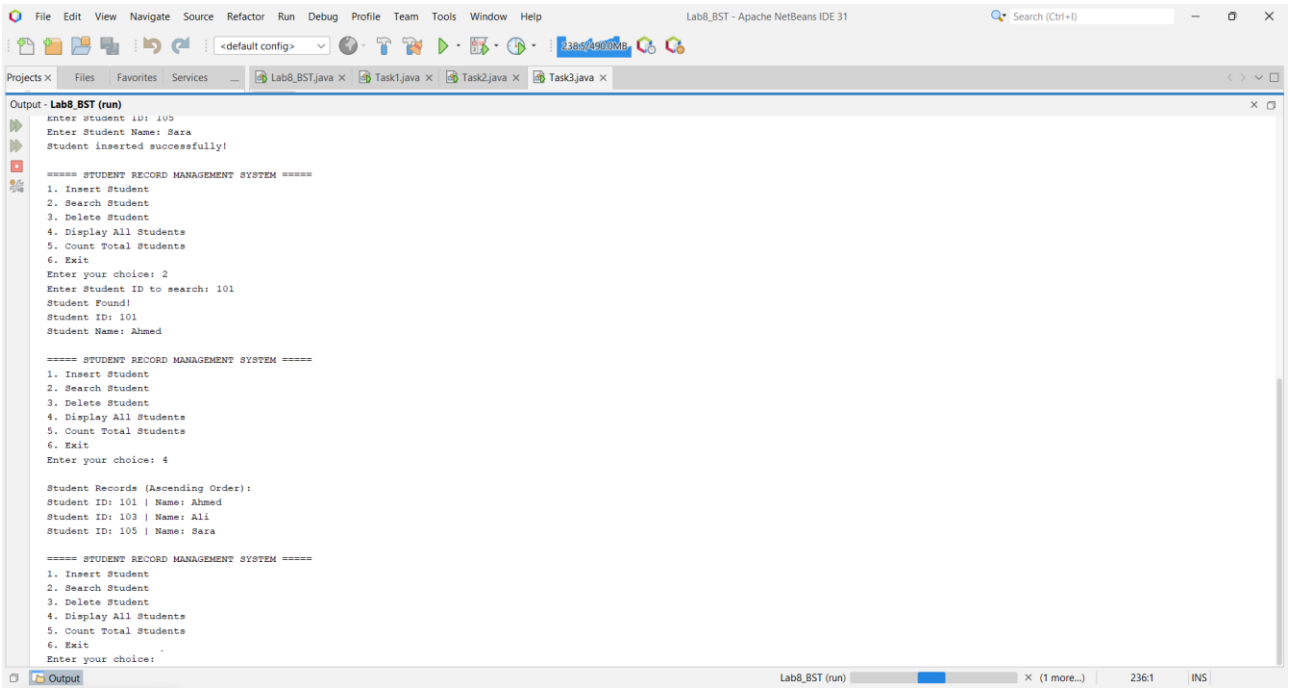


```
run:

===== STUDENT RECORD MANAGEMENT SYSTEM =====
1. Insert Student
2. Search Student
3. Delete Student
4. Display All Students
5. Count Total Students
6. Exit
Enter your choice: 1
Enter Student ID: 103
Enter Student Name: Ali
Student inserted successfully!

===== STUDENT RECORD MANAGEMENT SYSTEM =====
1. Insert Student
2. Search Student
3. Delete Student
4. Display All Students
5. Count Total Students
6. Exit
Enter your choice: 1
Enter Student ID: 101
Enter Student Name: Ahmed
Student inserted successfully!

===== STUDENT RECORD MANAGEMENT SYSTEM =====
1. Insert Student
2. Search Student
3. Delete Student
4. Display All Students
5. Count Total Students
6. Exit
Enter your choice: 1
Enter Student ID: 105
Enter Student Name: Sara
Student inserted successfully!
```



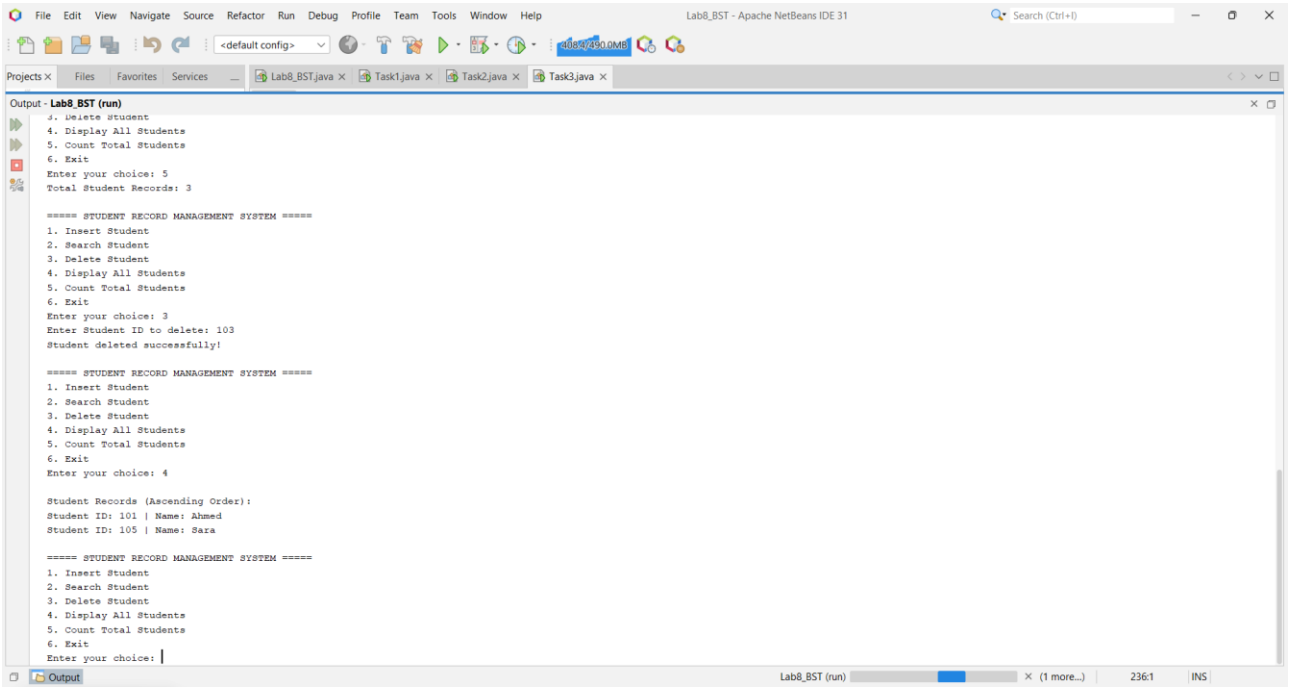
```
Enter Student ID: 105
Enter Student Name: Sara
Student inserted successfully!

===== STUDENT RECORD MANAGEMENT SYSTEM =====
1. Insert Student
2. Search Student
3. Delete Student
4. Display All Students
5. Count Total Students
6. Exit
Enter your choice: 2
Enter Student ID to search: 101
Student Found!
Student ID: 101
Student Name: Ahmed

===== STUDENT RECORD MANAGEMENT SYSTEM =====
1. Insert Student
2. Search Student
3. Delete Student
4. Display All Students
5. Count Total Students
6. Exit
Enter your choice: 4

Student Records (Ascending Order):
Student ID: 101 | Name: Ahmed
Student ID: 103 | Name: Ali
Student ID: 105 | Name: Sara

===== STUDENT RECORD MANAGEMENT SYSTEM =====
1. Insert Student
2. Search Student
3. Delete Student
4. Display All Students
5. Count Total Students
6. Exit
Enter your choice:
```



Github Link: https://github.com/Qayamraza85/Data-Struture-Lab/tree/main/lab8_bst